Projeto Final: um app completo, em equipe

Login, API, banco de dados local e navegação juntos em um único aplicativo, organizado nas camadas que estudamos.

Equipe: dupla ou trio **Valor:** 4,0 pontos **Entrega:** repositório no GitHub

De onde vem este projeto

Ao longo da etapa vocês aprenderam peças separadas: buscar dados na internet, guardar no aparelho, mudar de tela, organizar o código em pacotes. Em cada aula o app era pequeno e o exemplo já vinha escolhido. Agora a conversa é outra: **o app é de vocês**, e o desafio é fazer as peças funcionarem juntas.

É assim que um app real nasce: quase todo aplicativo pede que a pessoa entre com a sua conta, busca informação em um servidor, guarda parte dela no aparelho e deixa a pessoa circular por várias telas. O que muda é o assunto.

O que vocês vão construir

Um aplicativo Android em Kotlin com Jetpack Compose, escolhido pela equipe, que cumpra obrigatoriamente os quatro requisitos abaixo. O **tema é livre**: pode ser receitas, países, livros, personagens, clima, o que a equipe achar interessante, desde que exista uma API pública para alimentá-lo.

Requisito	O que precisa existir no app	Visto em aula
API REST	Dados vindos da internet com Retrofit, dentro de coroutines (nunca na Main Thread). A tela mostra o que está acontecendo: carregando, sucesso ou erro.	Retrofit e coroutines
Login e sessão	Telas de **cadastro** e de **login**. Só quem entrou vê o app. A pessoa continua logada ao fechar e reabrir, e um botão **Sair** volta ao login. Os dados salvos (favoritos, por exemplo) pertencem a cada usuário.	Room, estado e navegação juntos
Persistência local	Alguma informação guardada com Room (favoritos, histórico, itens salvos). Fechou e abriu o app, os dados continuam lá.	Room Database
Navegação	Navigation Compose com barra inferior de, no mínimo, 3 abas, e uma tela de detalhe que recebe o item escolhido por uma rota type-safe.	Navegação com o HabitTracker

Escolhendo a API

Escolham uma API **que não exija cadastro nem chave**, para não perder tempo com autenticação. Testem o endereço no navegador antes de decidir: se o JSON aparece, está pronta para usar. Algumas opções:

API	Endereço base	Ideia de uso com Room
PokéAPI	`pokeapi.co/api/v2`	Time de favoritos
TheMealDB	`themealdb.com/api/json/v1/1`	Livro de receitas salvas
Open Library	`openlibrary.org`	Lista de leitura
REST Countries	`restcountries.com/v3.1`	Países que quero visitar
Rick and Morty API	`rickandmortyapi.com/api`	Personagens favoritos
Open-Meteo	`api.open-meteo.com/v1`	Cidades salvas

Se a equipe preferir outra API, tudo bem, desde que o professor aprove antes de começar a codar.

Como organizar o código

O projeto precisa seguir a estrutura de camadas da aula de pacotes. A ideia é simples: cada arquivo mora na pasta da sua responsabilidade, e as dependências só andam em um sentido.

```
com.seuapp
 ├─ network       interface do Retrofit e a instância do Retrofit
 ├─ data          data classes, Room (Entity, Dao, Database) e Repository
 ├─ viewmodel     ViewModels e seus estados
 └─ userinterface telas Compose, navegação e MainActivity
```

- Nenhuma tela chama o Retrofit ou o DAO diretamente. Quem fala com eles é o **Repository**, em `data`.
- O ViewModel não importa nada de `userinterface`.
- Usuários seguem a mesma regra: `Usuario` (Entity) e `UsuarioDao` em `data`; a lógica de cadastrar e entrar fica no Repository, nunca dentro da tela.
- Só crie os pacotes que o app realmente usa.
- Troquem o `com.seunome` do projeto pelo nome de vocês já na criação (`namespace` e `applicationId`), senão o app abre e fecha na hora.

> **Como conferir a Regra de Dependência:** use a busca global (Ctrl+Shift+F) por `import retrofit2` dentro de `userinterface` e por `import ...userinterface` dentro de `viewmodel`. As duas buscas precisam voltar vazias.

> **Sobre o login deste projeto.** Ele é **local**: os usuários ficam no Room do próprio aparelho, o que serve para aprender o fluxo. Em um app de verdade, quem confere a senha é um servidor. Mesmo assim, **nunca salve a senha em texto puro**: guarde só o hash.
>
> ```
> fun String.sha256(): String =
>     java.security.MessageDigest.getInstance("SHA-256")
>         .digest(toByteArray()).joinToString("") { "%02x".format(it) }
> ```
>
> Regras mínimas: e-mail com formato válido, senha com 6 ou mais caracteres, e-mail único no cadastro, mensagem clara quando o login falha (sem dizer qual dos dois campos errou).

Sugestão de caminho

Não existe ordem obrigatória, mas equipes que seguem uma sequência parecida costumam ter menos retrabalho. Cada passo deixa algo funcionando para o próximo se apoiar.

1. **Planejem no papel.** Tema, API, quais serão as 3 abas, qual dado será salvo no Room, como será o login e a tela de detalhe. Desenhem as telas à mão, em caixinhas.
2. **Montem a base.** Projeto novo com nome da equipe, dependências, plugin de serialization e pacotes criados. Rodem no emulador antes de escrever qualquer tela.
3. **Tragam os dados.** Interface do Retrofit, data classes e Repository. Primeiro vejam a resposta chegar no Logcat, depois mostrem em uma lista.
4. **Guardem no aparelho.** Entity, Dao e Database do Room, ligados ao Repository.
5. **Construam o acesso.** Tabela de usuários no Room, telas de cadastro e login, validação dos campos e sessão guardada.
6. **Costurem a navegação.** Fluxo login para o app, barra inferior, rotas type-safe e tela de detalhe.
7. **Dêem acabamento.** Estados de carregamento e erro, textos revisados, README com prints, rodada de testes com a checklist.

> **Divisão em dupla:** uma pessoa cuida de `network` e `data` (API, Room, Repository, usuários) e a outra de `viewmodel` e `userinterface` (telas e navegação). Antes de separar, combinem quais funções do ViewModel serão o contrato entre vocês. Em trio, a terceira pessoa assume o login e a sessão. **Todos fazem commits.**

Ajuda da IDE

Três ferramentas do Android Studio resolvem a maior parte dos problemas de projeto:

Ferramenta	Para que serve neste projeto
Logcat (aba inferior)	Ver erros e mensagens. Filtre por `package:mine` e pelo nível Error. A linha `Caused by` costuma apontar a causa real.
App Inspection > Network Inspector	Ver cada chamada do Retrofit: endereço, código de resposta e JSON recebido. Ótimo para saber se o erro é da API ou do seu código.
App Inspection > Database Inspector	Ver as tabelas do Room em tempo real com o app rodando. Cadastrou um usuário? Salvou um favorito? As linhas precisam aparecer aqui. A coluna da senha **não pode** mostrar a senha em texto puro.

Erros comuns

Sintoma	Causa provável e saída
A requisição falha sem mensagem clara	Falta `<uses-permission android:name="android.permission.INTERNET"/>` no AndroidManifest.
O JSON chega, mas os campos vêm vazios ou dá erro de parsing	O nome do campo na data class não bate com o do JSON. Use `@SerialName("nome_no_json")` e confira no Network Inspector.
Erro ao acessar o banco pela tela	O Room não pode rodar na Main Thread. Chame o DAO por funções `suspend` dentro de `viewModelScope.launch`.
Mudei a Entity e o app quebrou ao abrir	O banco antigo ficou no aparelho com a estrutura velha. Desinstale o app do emulador ou aumente a versão do banco.
Fecho o app e ele volta para a tela de login	A sessão está só em `remember` ou no ViewModel. Guarde o id do usuário logado em `SharedPreferences` e leia na abertura.
`ClassNotFoundException` para a MainActivity	`namespace` e `applicationId` não batem com o pacote onde a MainActivity está.

Entrega

Enviem ao professor o link de um **repositório no GitHub** com um `README.md` neste formato:

```
# Nome do App
Uma frase dizendo o que o app faz.

## API utilizada
Nome, endereço base e o que usamos dela.

## Prints
(4 a 6 imagens: login, cadastro, lista, detalhe, favoritos, estado de erro)

## Divisão de tarefas
| Integrante | O que fez |
|------------|-----------|

## O que aprendemos
Uma frase de cada integrante sobre a sua parte.
```

Cada integrante também preenche, individualmente, o **formulário de feedback** da última página.

Como será avaliado

O professor vai instalar e usar o app, não só ler o código. A rotina de teste é: criar uma conta, tentar entrar com a senha errada, entrar corretamente, navegar por todas as abas, abrir um detalhe, salvar algo no Room, ligar o modo avião para ver o estado de erro, fechar o app e abrir de novo (precisa continuar logado, com o dado salvo), sair e entrar com outra conta (os dados não podem se misturar). Depois, olhar a organização dos pacotes e o README.

Critério	Pts	O que será verificado
Consumo de API (Retrofit e coroutines)	**0,7**	A requisição funciona, roda fora da Main Thread e a tela trata carregamento e erro.
Login e sessão	**0,7**	Cadastro e login com validação, senha com hash, sessão que sobrevive a fechar o app e botão Sair.
Persistência local (Room)	**0,7**	Os dados salvos sobrevivem a fechar e reabrir o app e pertencem ao usuário logado.
Navegação (fluxo de login, barra inferior e parâmetro type-safe)	**0,7**	3 ou mais abas; o app só abre depois do login; a tela de detalhe recebe o item correto.
Organização em pacotes	**0,7**	Responsabilidades separadas e Regra de Dependência respeitada.
Processo de equipe	**0,5**	Formulário preenchido, divisão de tarefas no README e commits de todos os integrantes.

Cada critério vale o **valor integral** quando atendido por completo, **metade** quando funciona com uma falha identificada e **zero** quando não foi atendido.

Antes de entregar

- [ ] O app abre no emulador sem travar e eu percorri todas as telas.
- [ ] Os dados da API aparecem, com indicação de carregamento e de erro.
- [ ] Cadastro e login funcionam, a senha errada mostra erro e a senha no banco é um hash.
- [ ] Fechei e reabri o app: continuo logado e os dados do Room continuam lá.
- [ ] Sair leva ao login, e outra conta não enxerga os dados da primeira.
- [ ] A tela de detalhe abre a partir da lista e mostra o item certo.
- [ ] As buscas da Regra de Dependência voltam vazias.
- [ ] O README tem prints, divisão de tarefas e uma frase de cada integrante.
- [ ] Todos os integrantes têm commits no repositório.

Feedback da equipe

Preencha sozinho e com sinceridade. As respostas ajudam o professor a enxergar como o trabalho aconteceu por dentro da equipe.

Seu nome:

Integrantes da equipe:

---

1. Como foi a participação de cada colega?

Colega	Nota de 1 a 5	Ponto forte	O que poderia melhorar

2. Sua própria atuação (1 = nunca, 5 = sempre)

Pergunta	1 a 5
Cumpri o que combinamos na equipe?	
Avisei das minhas dificuldades e pedi ajuda quando precisei?	
Ouvi as ideias dos colegas, mesmo quando eram diferentes das minhas?	

3. Um ponto positivo e um ponto a melhorar na minha atuação:
